

moneta (!)

Esercizio 1 (10 punti) Realizzare una funzione `union(A1,A2,n)` che dati due array di interi `A1` e `A2`, organizzati a max-heap, con capacità `n`, restituisce un nuovo array `A`, ancora organizzato a max-heap con capacità `2n`, che contiene l'unione insiemistica dei valori contenuti in `A1` e `A2`. Si assuma che `A1` e `A2` non contengano duplicati e si faccia in modo anche l'array ottenuto come unione non contenga duplicati. Ad es. se `A1` contiene i valori 3,1,2 e `A2` contiene i valori 5,2 allora l'unione `A` conterrà i valori 5,3,1,2, possibilmente non in questo ordine, ovvero l'elemento 2 non è duplicato. Valutare la complessità della funzione definita.

(Qualora il risultato `A` potesse contenere duplicati ci sarebbero soluzioni più efficienti?) → Dopo...

MAX

EXTRACT-MAX

$A_1 \rightarrow (3, 1, 2)$
 $A_2 \rightarrow (5, 2)$
 $(5, 3, 2, 1, 2)$

MAX(MIN) $\rightarrow O(n)$

EXTRACTMAX $\rightarrow$

MAX +

HEAPIFY ($\log(n)$)

`union(A1, A2, n)`

`allocate A[1..2n]`
`i = 0`

`//heapsize ..`

`while (A1.heapsize > 0) and (A2.heapsize > 0)`

`v1 = Max(A1)`

`v2 = Max(A2)`

`if(v1 == v2)`

`i++`

`A[i] = v1 // oppure v2`

`ExtractMax(A1)`

`ExtractMax(A2)`

`if(v1 < v2)`

`i = i + 1`

`A[i] = v1`

`ExtractMax(A1)`

`if(v1 > v2)`

`i = i + 1`

`A[i] = v2`

`ExtractMax(A2)`

`A.heapsize = i`

`return A`

$A_1 \rightarrow 3, 1, 2$

$A_2 \rightarrow 5, 2$

$A \rightarrow 5, 3, 1, 2, 2$

$A = A_1 \cup A_2$

✓

GLAD UNICO!

```
1 union_con_duplicati(A1, A2, n)
2   allocate A[1..2n]
3   for i = 1 to A1.heapsize
4     A[i] = A1[i]
5   for j = 1 to A2.heapsize
6     A[A1.heapsize + j] = A2[j]
7   A.heapsize = 2n
8   for i = [2n/2] downto 1
9     MaxHeapify(A, i)
10  return A
```

Complessità: $\Theta(n)$

Cosa fa: copia tutti gli elementi senza confrontarli (duplicati inclusi), poi BuildMaxHeap.

Esercizio 1 (10 punti) Siano dati due array $A[1..2n]$ e $B[1..n]$ organizzati a max-heap, entrambi contenenti n elementi ($\text{heapsize}=n$). Realizzare una procedura $\text{SortJoin}(A, B, n)$ che dati in input array A e B con le proprietà sopra descritte, ritorna in A un array ordinato contenente tutti i $2n$ elementi originariamente presenti in A e B . L'array B può essere modificato durante l'esecuzione della procedura, se necessario, ma l'algoritmo dovrà operare in *spazio costante*. Dare lo pseudocodice della procedura, motivarne la correttezza e valutarne la complessità. Se si utilizzano operazioni sui max-heap andranno definite esplicitamente.

$A [1 \dots 2N]$
 $B [1 \dots N]$

$\rightarrow S = [1 \dots 2N]$
 $\rightarrow \text{ORDINATI}$
 SPAZIO COSTANTE

```

SortJoin(A, B, n)
  // A ha lunghezza 2n, ma heapsize(A)=n (elementi validi in A[1..n])
  // B ha lunghezza n, heapsize(B)=n

  // 1) Copia B negli slot liberi di A
  for i = 1 to n do
    A[n + i] = B[i]

  // 2) Costruisci un max-heap su 2n elementi
  heapsize(A) = 2*n
  BuildMaxHeap(A)

  // 3) HeapSort in-place su A[1..2n]
  HeapSort(A)
return A

HEAPSORT(A)
1 BUILD-MAX-HEAP(A) // O(n)
2 for i = A.length down to 2
3   A[1] ↔ A[i]
4   A.heapsize = A.heapsize - 1
5   MAX-HEAPIFY(A, 1) // O(log n)

BUILD-MAX-HEAP(A)
1 A.heapsize = A.length
2 for i = ⌊A.length/2⌋ down to 1
3   MAX-HEAPIFY(A, i)

MAX-HEAPIFY(A, i)
1 l = LEFT(i)
2 r = RIGHT(i)
3 if (l ≤ A.heapsize) and (A[l] > A[i])
4   max = l
5 else
6   max = i
7 if (r ≤ A.heapsize) and (A[r] > A[max])
8   max = r
9 if (max ≠ i)
10  A[i] ↔ A[max]
11  MAX-HEAPIFY(A, max)
  
```

Esercizio 1 (10 punti) Un ricco possidente deve lasciare in eredità le sue $2n$ proprietà a n figli. Il valore delle proprietà è contenuto in un array di numeri (reali positivi) $A[1..2n]$.

1. Sviluppare un algoritmo $\text{Split}(A, n)$ che dato in input l'array $A[1..2n]$ dei valori delle proprietà e numero n , verifica se le $2n$ proprietà possano partizionate in n coppie, tutte con lo stesso valore complessivo (di modo da assegnare una coppia di proprietà a ciascun figlio). Si assuma che $n > 0$.
2. Si valuti la complessità dell'algoritmo proposto.
3. Si dimostri la correttezza dell'algoritmo proposto.

```

1. Split(A, n)
2.   MergeSort(A, 1, 2n)           // ordina array
3.   S = 0
4.   for i = 1 to 2n do
5.     S = S + A[i]                 // calcola somma totale
6.   if S mod n ≠ 0 then
7.     return FALSO
8.   t = S / n                       // target per ogni coppia
9.   return SplitRec(A, 1, 2n, t)

10. SplitRec(A, p, r, t)           // verifica A[p..r]
11.   if p > r then                  // caso base: array vuoto
12.     return VERO
13.   if p = r then                 // caso base: elemento singolo
14.     return FALSO               // impossibile formare coppia
15.   if A[p] + A[r] = t then       // estremi formano coppia valida
16.     return SplitRec(A, p+1, r-1, t) // ricorri su sottoarray
17.   else
18.     return FALSO               // nessuna coppia valida possibile
  
```

Domanda B (6 punti) Scrivere la ricorrenza sulle lunghezze $\ell(i, j)$ per il problema della longest common subsequence (LCS).

Definisco

$$\ell(i, j) = |LCS(X_i, Y_j)|$$
$$\ell(i, j) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 & (\text{caso 0}) \\ \ell(i-1, j-1) + 1 & \text{se } i, j > 0 \text{ e } x_i = x_j & (\text{caso 1}) \\ \max\{\ell(i, j-1), \ell(i-1, j)\} & \text{se } i, j > 0 \text{ e } x_i \neq x_j & (\text{caso 2}) \end{cases}$$

ANALISI DEL PROBLEMA

Input: Array $A[1..2n]$ di valori reali positivi, intero $n > 0$

Output: VERO se le $2n$ proprietà possono essere partizionate in n coppie con stesso valore complessivo, FALSO altrimenti

Condizioni necessarie:

1. Somma totale S deve essere divisibile per n
2. Ogni coppia deve sommare esattamente $t = S/n$
3. Deve esistere un matching perfetto con questa proprietà